

# Module 13 Structured Notes: Linear vs Looping Lists

## 1. Current Progress Analysis

You are now at **Module 13**.

Before this module, you already studied the main ideas needed for singly linked lists:

| Previous topic          | Why it matters in Module 13                                                              |
|-------------------------|------------------------------------------------------------------------------------------|
| Arrays vs linked lists  | You understand why linked lists use pointers instead of direct indexes.                  |
| Node structure          | You know that each node has <code>data</code> and <code>next</code> .                    |
| Traversal               | You know how to move through a list using <code>p = p-&gt;next</code> .                  |
| <code>NULL</code>       | You know that a normal linked list ends when the last node points to <code>NULL</code> . |
| Insertion and deletion  | You know how links are changed between nodes.                                            |
| Reversal                | You know that pointer links can be changed carefully.                                    |
| Concatenation and merge | You know how multiple nodes and lists can be connected.                                  |

Module 13 uses all of these ideas.

The main new topic is:

How do we know whether a linked list ends normally or contains a loop?

## 2. Big Idea of Module 13

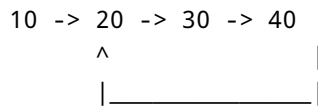
Until now, most linked lists looked like this:

```
10 -> 20 -> 30 -> 40 -> NULL
```

This is called a **linear linked list**.

It has a clear end because the last node points to `NULL`.

But sometimes a node may point back to an earlier node:



This is called a **looping linked list** or a **cyclic linked list**.

In this module, you learn:

1. What a linear list is.
2. What a looping list is.
3. Why loops are dangerous for normal traversal.
4. How to manually create a loop for testing.
5. How to detect a loop using Floyd's cycle detection algorithm.
6. Why the detection code uses a `do-while` loop.
7. How to dry-run loop detection.
8. The time and space complexity of loop detection.

## 3. Linear Linked List

### 3.1 Meaning

A **linear linked list** is a linked list that ends at `NULL`.

Example:

```
first
|
v
10 -> 20 -> 30 -> 40 -> NULL
```

The last node is `40`.

Its `next` pointer stores `NULL`.

That means:

There is no node after 40.  
The list ends here.

---

## 3.2 Traversing a Linear List

A normal traversal starts from the first node and moves one node at a time.

Example:

```
while (p != NULL) {  
    printf("%d ", p->data);  
    p = p->next;  
}
```

This works safely for a linear list because `p` eventually becomes `NULL`.

Dry run:

```
p points to 10 -> print 10  
p points to 20 -> print 20  
p points to 30 -> print 30  
p points to 40 -> print 40  
p points to NULL -> stop
```

Output:

```
10 20 30 40
```

---

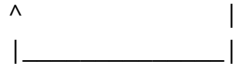
## 4. Looping Linked List

### 4.1 Meaning

A **looping linked list** is a linked list where a node points back to an earlier node instead of reaching `NULL`.

Example:

```
first  
|  
v  
10 -> 20 -> 30 -> 40 -> 50
```



Here:

50->next points back to 30

So after node `50`, the list goes back to node `30`.

## 4.2 What Happens During Traversal

Normal traversal becomes:

10 -> 20 -> 30 -> 40 -> 50 -> 30 -> 40 -> 50 -> 30 -> ...

The traversal never reaches `NULL`.

That means the list does not have a normal ending point.

## 4.3 Why Looping Lists Are Dangerous

This normal display function works for a linear list:

```
void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

But if the list has a loop, this function may run forever.

Why?

Because the condition is:

`p != NULL`

In a looped list, `p` may never become `NULL`.

So the program may keep printing forever.

---

## 5. Linear List vs Looping List

| Feature           | Linear list                 | Looping list                   |
|-------------------|-----------------------------|--------------------------------|
| Ending            | Ends at <code>NULL</code>   | Does not end normally          |
| Last node         | Points to <code>NULL</code> | Points back to an earlier node |
| Normal traversal  | Safe                        | Dangerous                      |
| Display function  | Stops normally              | May run forever                |
| Detection needed? | No problem if known         | Yes, must detect loop          |

---

## 6. Floyd's Cycle Detection Algorithm

### 6.1 Big Idea

To detect a loop, Module 13 uses **Floyd's cycle detection algorithm**.

It is also called:

slow and fast pointer method

The idea is simple:

Use two pointers.

| Pointer        | Movement                | Meaning      |
|----------------|-------------------------|--------------|
| <code>p</code> | Moves 1 step at a time  | Slow pointer |
| <code>q</code> | Moves 2 steps at a time | Fast pointer |

---

### 6.2 Simple Analogy

Imagine two runners on a track.

One runner is slow.

One runner is fast.

If the track is circular, the fast runner will eventually catch the slow runner.

In a linked list:

- If the list has no loop, the fast pointer reaches `NULL`.
- If the list has a loop, the fast pointer eventually meets the slow pointer.

---

## 6.3 What Means “Loop Found”?

If the two pointers meet, then a loop exists.

In code:

```
p == q
```

This means:

```
Both pointers are pointing to the same node.
```

If this happens after the pointers have started moving, the list contains a loop.

---

## 7. Visual Idea of Floyd Detection

Suppose the list is:

```
10 -> 20 -> 30 -> 40 -> 50
                ^           |
                |_____||
```

The slow pointer `p` moves like this:

```
10, 20, 30, 40, 50, 30, 40, ...
```

The fast pointer `q` moves like this:

```
10, 30, 50, 40, 30, 50, ...
```

Because `q` moves faster, it eventually reaches the same node as `p`.

When:

```
p == q
```

we know there is a loop.

---

## 8. Why `do-while` Is Used

### 8.1 The Beginner Trap

At the start of the algorithm, both pointers begin at the same node:

```
p = q = first;
```

So immediately:

```
p == q
```

This is true before any movement happens.

If we wrote this:

```
while (p != q) {  
    // move pointers  
}
```

The loop would not run even once.

Why?

Because `p` and `q` are already equal at the beginning.

That would make the algorithm fail.

---

## 8.2 Correct Idea

We need the pointers to move first.

Then we check whether they meet.

That is why we use a `do-while` loop.

A `do-while` loop runs the body first, then checks the condition.

Basic structure:

```
do {  
    // move pointers first  
} while (p != NULL && q != NULL && p != q);
```

This gives both pointers a chance to move away from the starting node.

---

## 9. Loop Detection Function

```
int isLoop(struct Node *f) {  
    struct Node *p;  
    struct Node *q;  
  
    p = q = f;  
  
    if (f == NULL) {  
        return 0;  
    }  
  
    do {  
        p = p->next;    // slow pointer moves 1 step  
        q = q->next;    // fast pointer moves first step  
  
        if (q != NULL) {  
            q = q->next; // fast pointer moves second step  
        }  
  
    } while (p != NULL && q != NULL && p != q);  
  
    if (p == q) {  
        return 1;    // loop found  
    } else {
```



```
        return 0;    // no loop found
    }
}
```

## 10. Explanation of the Code

### 10.1 Function Header

```
int isLoop(struct Node *f)
```

Meaning:

| Part                        | Meaning                                |
|-----------------------------|----------------------------------------|
| <code>int</code>            | The function returns an integer.       |
| <code>isLoop</code>         | Function name.                         |
| <code>struct Node *f</code> | Pointer to the first node of the list. |

Return values:

| Return value   | Meaning       |
|----------------|---------------|
| <code>1</code> | Loop found    |
| <code>0</code> | No loop found |

### 10.2 Create Two Pointers

```
struct Node *p;  
struct Node *q;
```

These are the two pointers used for detection.

| Pointer        | Role         |
|----------------|--------------|
| <code>p</code> | Slow pointer |
| <code>q</code> | Fast pointer |

## 10.3 Start Both at the First Node

```
p = q = f;
```

Both pointers start at the first node.

Example:

```
first
|
v
10 -> 20 -> 30 -> 40 -> NULL
^
|
p, q
```

---

## 10.4 Empty List Check

```
if (f == NULL) {
    return 0;
}
```

If the list is empty, there is no node.

So there cannot be a loop.

Return:

```
0
```

---

## 10.5 Move Slow Pointer

```
p = p->next;
```

This moves `p` one step forward.

Example:

```
Before: p -> 10  
After:  p -> 20
```

---

## 10.6 Move Fast Pointer First Step

```
q = q->next;
```

This moves `q` one step forward.

---

## 10.7 Move Fast Pointer Second Step Safely

```
if (q != NULL) {  
    q = q->next;  
}
```

This is important.

We only move `q` again if `q` is not `NULL`.

Why?

Because this would be unsafe:

```
q = q->next;
```

if `q` is already `NULL`.

You cannot access `q->next` when `q` is `NULL`.

---

## 10.8 Loop Condition

```
while (p != NULL && q != NULL && p != q);
```

The loop continues while all three conditions are true:

| Condition              | Meaning                               |
|------------------------|---------------------------------------|
| <code>p != NULL</code> | Slow pointer has not reached the end. |
| <code>q != NULL</code> | Fast pointer has not reached the end. |
| <code>p != q</code>    | The two pointers have not met.        |

The loop stops when one of these happens:

1. `p` becomes `NULL`.
2. `q` becomes `NULL`.
3. `p` and `q` meet.

## 10.9 Final Check

```
if (p == q) {
    return 1;
} else {
    return 0;
}
```

If `p == q`, the two pointers met.

That means the list has a loop.

If they did not meet, one pointer reached `NULL`.

That means the list is linear.

## 11. Dry Run 1: Linear List

### 11.1 List

```
10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

Start:

```
p = 10
q = 10
```

## 11.2 Pointer Movement

| Iteration | p moves 1 step | q moves 2 steps | Result    |
|-----------|----------------|-----------------|-----------|
| 1         | 20             | 30              | Not equal |
| 2         | 30             | 50              | Not equal |
| 3         | 40             | NULL            | Stop      |

## 11.3 Conclusion

The fast pointer `q` reaches `NULL`.

So the list has no loop.

Return:

```
0
```

Meaning:

```
No loop found.
```

## 12. Dry Run 2: Looping List

### 12.1 List

```
10 -> 20 -> 30 -> 40 -> 50
                ^         |
                |_____|
```

Here:

```
50->next points back to 30
```

Start:

```
p = 10  
q = 10
```

## 12.2 Pointer Movement

| Iteration | p moves 1 step | q moves 2 steps | Result    |
|-----------|----------------|-----------------|-----------|
| 1         | 20             | 30              | Not equal |
| 2         | 30             | 50              | Not equal |
| 3         | 40             | 40              | Equal     |

## 12.3 Conclusion

Now:

```
p == q
```

Both pointers point to node 40 .

So the list has a loop.

Return:

```
1
```

Meaning:

```
Loop found.
```

---

# 13. How to Manually Create a Loop for Testing

## 13.1 Normal List

Suppose we first create this normal list:

```
10 -> 20 -> 30 -> 40 -> 50 -> NULL
```

Now we want to create this loop:

```
10 -> 20 -> 30 -> 40 -> 50
                ^           |
                |_____|
```

This means:

50 should point back to 30.

In code:

```
50->next = address of node 30;
```

---

## 13.2 Pointers Needed

We need two temporary pointers:

| Pointer | Meaning           |
|---------|-------------------|
| t1      | Points to node 30 |
| t2      | Points to node 50 |

Code:

```
struct Node *t1;
struct Node *t2;

t1 = first->next->next;           // points to 30
t2 = first->next->next->next->next; // points to 50

t2->next = t1;                   // creates loop
```

After this line:

```
t2->next = t1;
```

node `50` points back to node `30`.

The list no longer ends at `NULL`.

---

## 13.3 Important Warning

After creating a loop, do not call normal display:

```
display(first);
```

This may run forever.

Use a limited display if you need to print a few nodes for testing.

---

## 14. Limited Display for Testing

A normal display function can run forever if the list has a loop.

So for testing, we can use a limited display function.

```
void displayLimited(struct Node *p, int limit) {
    int count = 0;

    while (p != NULL && count < limit) {
        printf("%d ", p->data);
        p = p->next;
        count++;
    }

    if (p != NULL) {
        printf("...");
    }
}
```

### 14.1 Meaning

This function prints only a fixed number of nodes.



Example call:

```
displayLimited(first, 12);
```

This means:

```
Print at most 12 nodes.
```

If the list still has more nodes after the limit, it prints:

```
...
```

This helps us avoid infinite output.

---

## 15. Full C Program for Module 13

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
```

```

        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

int isLoop(struct Node *f) {
    struct Node *p;
    struct Node *q;

    p = q = f;

    if (f == NULL) {
        return 0;
    }

    do {
        p = p->next;
        q = q->next;

        if (q != NULL) {
            q = q->next;
        }

    } while (p != NULL && q != NULL && p != q);

    if (p == q) {
        return 1;
    } else {
        return 0;
    }
}

void displayLimited(struct Node *p, int limit) {
    int count = 0;

    while (p != NULL && count < limit) {
        printf("%d ", p->data);
        p = p->next;
        count++;
    }

    if (p != NULL) {
        printf("...");
    }
}

```

```

}

int main() {
    int A[] = {10, 20, 30, 40, 50};

    create(A, 5);

    printf("Before creating loop:\n");
    displayLimited(first, 10);

    if (isLoop(first)) {
        printf("\nLoop found\n");
    } else {
        printf("\nNo loop found\n");
    }

    // Manually create loop: 50 points back to 30
    struct Node *t1 = first->next->next;           // 30
    struct Node *t2 = first->next->next->next->next; // 50

    t2->next = t1;

    printf("\nAfter creating loop:\n");
    displayLimited(first, 12);

    if (isLoop(first)) {
        printf("\nLoop found\n");
    } else {
        printf("\nNo loop found\n");
    }

    return 0;
}

```

## 16. Expected Output

Conceptually, the output is:

```

Before creating loop:
10 20 30 40 50
No loop found

After creating loop:

```

```
10 20 30 40 50 30 40 50 30 40 50 30 ...  
Loop found
```

The first test says:

```
No loop found
```

because the list ends at `NULL`.

The second test says:

```
Loop found
```

because node `50` points back to node `30`.

---

## 17. Complexity Analysis

### 17.1 Loop Detection Time Complexity

Floyd's loop detection is:

```
O(n)
```

Why?

Because the pointers move through the list nodes.

In the worst case, they may need to move through many nodes before either:

1. reaching `NULL`, or
2. meeting inside the loop.

So the time grows with the number of nodes.

---

### 17.2 Loop Detection Space Complexity

The space complexity is:

$O(1)$

Why?

Because the algorithm uses only two main pointers:

p  
q

It does not use an array.

It does not create extra nodes.

It does not use a hash table.

So the extra memory stays constant.

---

## 17.3 Manual Loop Creation Complexity

If we already have pointers to the two needed nodes, loop creation is:

$O(1)$

Because it only changes one link:

```
t2->next = t1;
```

But if we must search for `t1` and `t2` first, then finding those nodes may take:

$O(n)$

---

## 18. Beginner Mistakes to Avoid

### Mistake 1: Using Normal Display on a Looped List

Wrong:

```
display(first);
```

Problem:

If the list has a loop, this may never stop.

Better:

```
displayLimited(first, 20);
```

---

## Mistake 2: Moving Fast Pointer Without Checking `NULL`

Wrong:

```
q = q->next->next;
```

Problem:

If `q->next` is `NULL`, this can crash.

Safer:

```
q = q->next;

if (q != NULL) {
    q = q->next;
}
```

---

## Mistake 3: Using `while (p != q)` at the Start

Wrong:

```
p = q = first;

while (p != q) {
```

```
    // move pointers
}
```

Problem:

At the start, `p` and `q` are already equal.

So the loop does not run.

Correct:

```
do {
    // move pointers
} while (p != NULL && q != NULL && p != q);
```

## Mistake 4: Thinking Reverse Display Is a Loop

Reverse display from recursion prints values backward.

It does not create a loop.

A loop means a node's `next` pointer points back to an earlier node.

Example of loop:

50->next points back to 30

## Mistake 5: Thinking a Loop Must Start at the First Node

A loop can start anywhere.

Example:

```
10 -> 20 -> 30 -> 40 -> 50
                ^           |
                |_____||
```

The loop starts at `30`, not at `10`.

The first few nodes can still be outside the loop.

---

## 19. Key Code Patterns to Memorize

### 19.1 Normal Traversal

```
while (p != NULL) {  
    p = p->next;  
}
```

Use this only when the list is known to be linear.

---

### 19.2 Manual Loop Creation

```
t2->next = t1;
```

This makes one node point back to an earlier node.

---

### 19.3 Floyd Movement

```
p = p->next;  
q = q->next;  
  
if (q != NULL) {  
    q = q->next;  
}
```

`p` moves one step.

`q` moves two steps safely.

---

### 19.4 Detection Condition

```
p == q
```

If this becomes true after movement, a loop exists.



---

## 20. Practice Questions

### Question 1

What type of list is this?

5 -> 10 -> 15 -> NULL

Answer:

Linear list

Reason:

The list ends at NULL.

---

### Question 2

What type of list is this?

5 -> 10 -> 15 -> 20  
          ^          |  
          |\_\_\_\_\_|

Answer:

Looping list

Reason:

Node 20 points back to node 10.

---

### Question 3

Why is this unsafe?

```
q = q->next->next;
```

Answer:

Because `q->next` might be `NULL`.

If it is `NULL`, then accessing `q->next->next` is invalid.

---

## Question 4

Why do we use `do-while` in loop detection?

Answer:

Because `p` and `q` both start at the same node.

We need to move them first before checking whether they meet.

---

## Question 5

What does this mean?

```
if (p == q)
```

Answer:

Both pointers are pointing to the same node.

In Floyd's algorithm, that means a loop was found.

---

# 21. Final Summary

Module 13 teaches the difference between a normal linked list and a looped linked list.

A **linear linked list** ends at `NULL`:

```
10 -> 20 -> 30 -> NULL
```

A **looping linked list** points back to an earlier node:

```
10 -> 20 -> 30 -> 40
      ^           |
      |           |
      +-----> 20
```

A normal display function is unsafe for looped lists because it may never reach **NULL**.

To detect a loop, use Floyd's cycle detection method:

```
p moves 1 step
q moves 2 steps
```

If either pointer reaches **NULL**, the list is linear.

If **p == q**, the list has a loop.

The detection algorithm takes:

```
Time: O(n)
Space: O(1)
```

The most important idea is:

A loop exists when following **next** pointers causes you to visit the same nodes again instead of reaching **NULL**.